Optimizers in Deep Learning: Why Gradient Descent Needs Help

Contents

1. Overview 2
2. Prerequisites 3
3. What an optimizer does 3
4. Gradient descent, the optimizer so far 4
5. Five weak spots of plain gradient descent 6
    - 5.1 Choosing the learning rate 6
    - 5.2 Learning-rate schedules are fixed in advance 7
    - 5.3 One learning rate for every direction 7
    - 5.4 Local minima 11
    - 5.5 Saddle points 11
    - 5.6 What goes wrong as a result 13
6. The optimizers ahead 13
7. Summary 15
8. Sources 15
9. Key terms 15

Where this fits: Pipeline map step 8 (Model). See the Course map.

0 Foundations	1 Frame	2 Get data	3 Understand	4 Clean	5 Features	6 Reduce
7 Split	**8 Model**	9 Evaluate	10 Tune	11 Deploy	12 Test	13 Monitor

- **Builds on:** Gradient descent; Mini-batch gradient descent; Convex sets and convex optimisation; Backpropagation; Improving a neural network.
- **Leads to:** SGD with momentum; Nesterov accelerated gradient (NAG); AdaGrad; RMSProp; Adam.

1. Overview

Key point: An optimizer is the rule that turns gradients into weight updates. Plain gradient descent has five weak spots: choosing the learning rate, fixed **learning-rate schedules** (G-1070), one learning rate for every direction, local minima and saddle points. The improved optimizers fix them with two ideas: build up speed, and adapt the learning rate.

Training a deep network can take a long time, so a lot of work goes into making it faster. Three techniques already help:

- good weight initialisation (G-947);
- batch normalisation (G-266);
- the choice of activation function (G-165).

The fourth, and probably the most important for speed, is the **optimizer** (G-1401): the algorithm that finds the values of the weights and biases for which the loss is smallest.

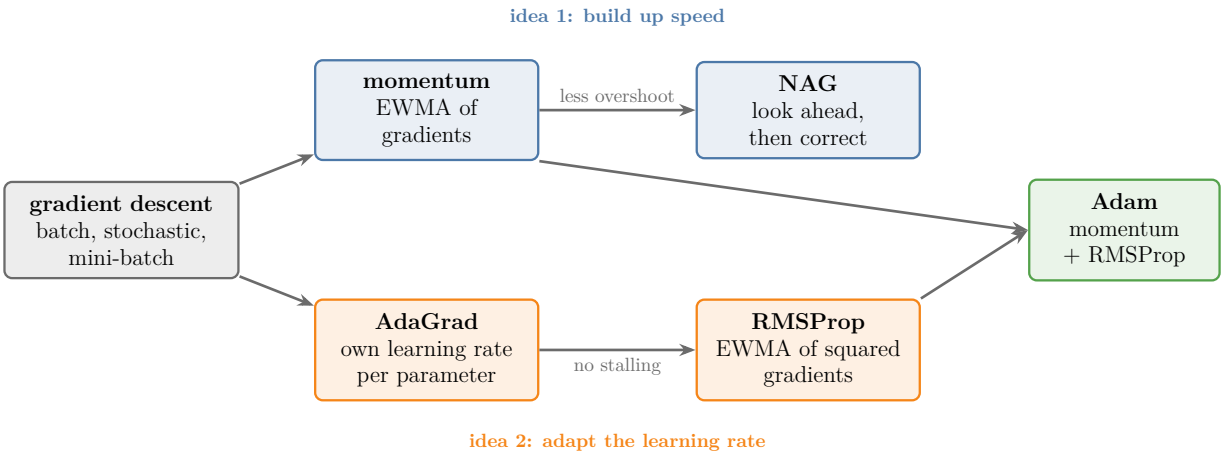


Figure 1: The optimizers ahead. Momentum and NAG build up speed; AdaGrad and RMSProp adapt the learning rate; Adam combines both

Figure 1 is the plan. Each optimizer is a small change to **gradient descent** (G-862), not

something new from scratch. Several of them rely on one tool, the **exponentially weighted moving average** (G-735), which comes first.

2. Prerequisites

- The loss as a function of all the weights and why the update subtracts the derivative.
- Batch, stochastic and mini-batch gradient descent in a network.
- The learning rate and the contour plot.

3. What an optimizer does

> **Key point:** Training is an optimisation problem: find the weights and biases that make the loss smallest. We start from random values and improve them step by step.

Take a classification task with two input **features** (G-772; input variables, the columns of the data table) and one **target** (G-1949; the output we predict). A network with one hidden layer of 2 nodes and one output node has 9 parameters. Each hidden node has one weight per input plus one bias; the output node has one weight per hidden node plus one bias (Notebook):

$$\text{hidden: } 2 \times 2 + 2 = 6$$

$$\text{output: } 2 + 1 = 3$$

$$\text{total: } 6 + 3 = 9$$

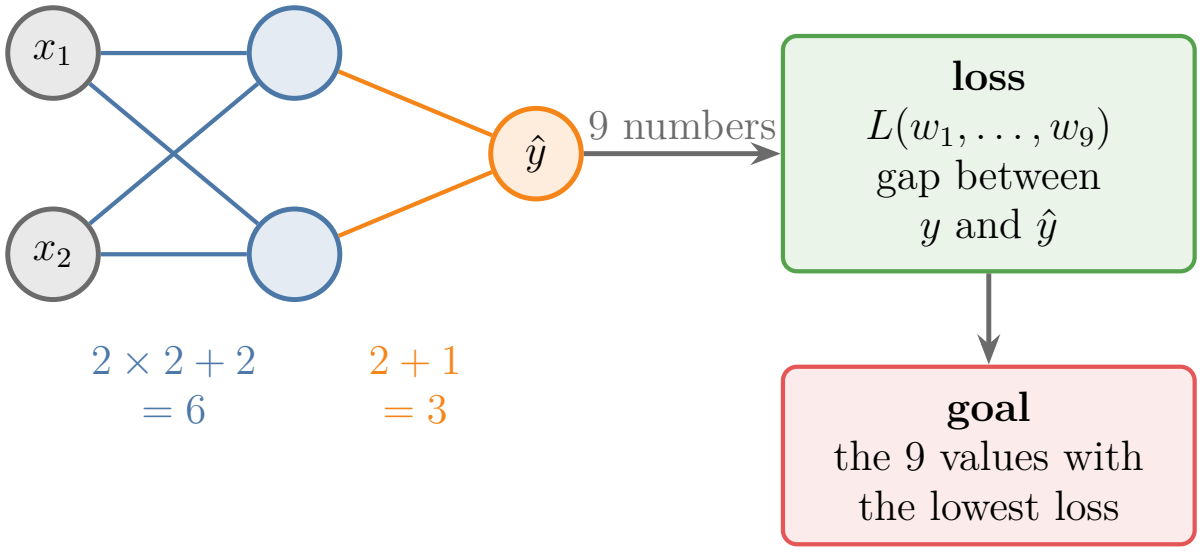


Figure 2: A 2-2-1 network has 9 weights and biases. The loss turns those 9 numbers into one score, and training searches for the 9 values with the lowest loss

Figure 2 shows the whole job of an **optimizer** (G-1401): 9 numbers go in, one loss comes out, and we want the lowest loss.

Training must find values of these 9 numbers for which the network's predictions $\hat{y}$ are as close as possible to the true values y. The loss measures the gap between y and $\hat{y}$, so training is an **optimisation problem** (G-1399): minimise the loss. The weights and biases at the minimum are the network's best parameters.

The loss is a function of all 9 parameters, so its graph lives in 10 dimensions, which we cannot draw. With only 2 weights it becomes a surface over the (w_1, w_2) plane: we start at a random point and walk downhill to the lowest point, the **global minimum** (G-848; the same picture for the loss of all nine parameters).

4. Gradient descent, the optimizer so far

> **Key point:** Gradient descent moves every weight a small step against its slope, again and again. Batch, stochastic and mini-batch gradient descent differ only in how many rows of the data they use for each step.

The **optimizer** (G-1401) used so far is **gradient descent** (G-862). In plain words: stand on the loss surface, feel which way is uphill, and take one small step the other way. One step on a small example, with two weights and the loss $L = w_1^2 + w_2^2$, starting at $w_1 = 3$, $w_2 = 4$ with step size $\eta = 0.1$:

$$L = 3^2 + 4^2 = 25$$

The slope in each weight's direction, with the other weight held fixed, is its **partial derivative** (G-1457; computing partial derivatives):

$$\frac{\partial L}{\partial w_1} = 2w_1 = 6$$

$$\frac{\partial L}{\partial w_2} = 2w_2 = 8$$

The list of these slopes, $(6, 8)$, is the **gradient** (G-863; collecting the partial derivatives), written $\nabla_w L$. Each weight steps against its slope:

$$w_1 = 3 - 0.1 \times 6 = 2.4$$

$$w_2 = 4 - 0.1 \times 8 = 3.2$$

$$L = 2.4^2 + 3.2^2 = 5.76 + 10.24 = 16$$

The loss fell from 25 to 16. Written for every weight at once, the step is:

$$w_{t+1} = w_t - \eta \, \nabla_w L(w_t)$$

where w_t is the list of weights after t steps, η (eta) is the **learning rate** (G-1068; how far to move), here 0.1, and $\nabla_w L$ is the gradient of the loss with respect to the weights. We repeat the update for a chosen number of **epochs** (G-696; full passes over the training data).

Gradient descent has three variants. They differ only in the number of rows (training examples) used for each update (batch, stochastic and mini-batch in a network). With 500 rows and 10 epochs:

Variant	Rows per update	Updates in 10 epochs
Batch	all 500	10
Stochastic (SGD)	1	5,000
Mini-batch, batch size 100	100	50

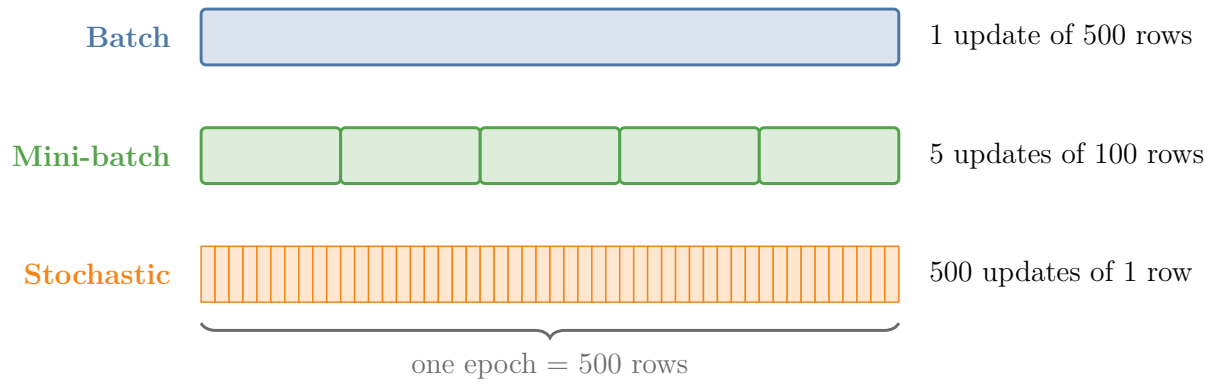


Figure 3: One epoch of 500 rows cut three ways. Batch makes 1 update, mini-batch 5, stochastic 500 (its strip is drawn with 50 slices for space)

Figure 3 shows that the three variants see the same rows per epoch; only the number of updates changes.

So we already have three optimizers. The next section explains why we need more.

5. Five weak spots of plain gradient descent

> **Key point:** Gradient descent is hard to tune (learning rate, schedule), treats every direction the same, and can get stuck at local minima and **saddle points** (G-1718). Training then becomes slow, or ends at a poor solution.

These five challenges are the standard list for plain mini-batch gradient descent (Ruder 2016, §3).

5.1 Choosing the learning rate

> **Key point:** Too small a learning rate converges painfully slowly; too large a one overshoots, oscillates or diverges. The right value depends on the data.

Every update subtracts η times the gradient, so η sets the size of every step (a learning rate too small or too large):

- **Too small:** every step is tiny and **convergence** (G-472; the steps settling at the minimum) is painfully slow; training may stop before reaching the minimum.
- **Too large:** the steps jump over the minimum, the path zigzags, and the loss can grow without limit.

A value in between works best, but finding it for a given dataset takes trial and error (Ruder 2016, §3).

5.2 Learning-rate schedules are fixed in advance

> **Key point:** A schedule lowers the learning rate during training, but its timetable is fixed before training starts, so it cannot adapt to the dataset.

A **learning-rate schedule** (G-1070) changes the learning rate during training, usually lowering it after a set number of epochs or when the loss stops improving by a threshold (as in the learning schedules of stochastic gradient descent). The schedule and the thresholds must be defined before training. Different datasets need different schedules, so a schedule that works well on one dataset may fail on another (Ruder 2016, §3).

5.3 One learning rate for every direction

> **Key point:** All parameters share one learning rate. When the loss is steep in one direction and flat in another, a rate safe for the steep direction is far too slow for the flat one.

With 9 parameters, the loss surface has 9 directions to move in. **Gradient descent** (G-862) uses the same η for all of them, so it descends at the same "speed setting" in every direction. We cannot give w_1 one learning rate and w_2 another.

The loss is often very sensitive to some directions and insensitive to others (Goodfellow et al. 2016, §8.5). Take a narrow valley,

$$L(w_1, w_2) = \tfrac{1}{2}\left(w_1^2 + 100\, w_2^2\right)$$

which is 100 times steeper across (w_2) than along (w_1). Its value at the starting point $(w_1, w_2) = (-10, 0.4)$ used below:

$$L(-10, 0.4) = \tfrac{1}{2}\left((-10)^2 + 100 \times 0.4^2\right)$$

$$= \tfrac{1}{2}(100 + 16)$$

$$= 58$$

Every pair (w_1, w_2) gives one loss, so the loss is a surface: the pair is the position on the floor, the loss is the height above it.

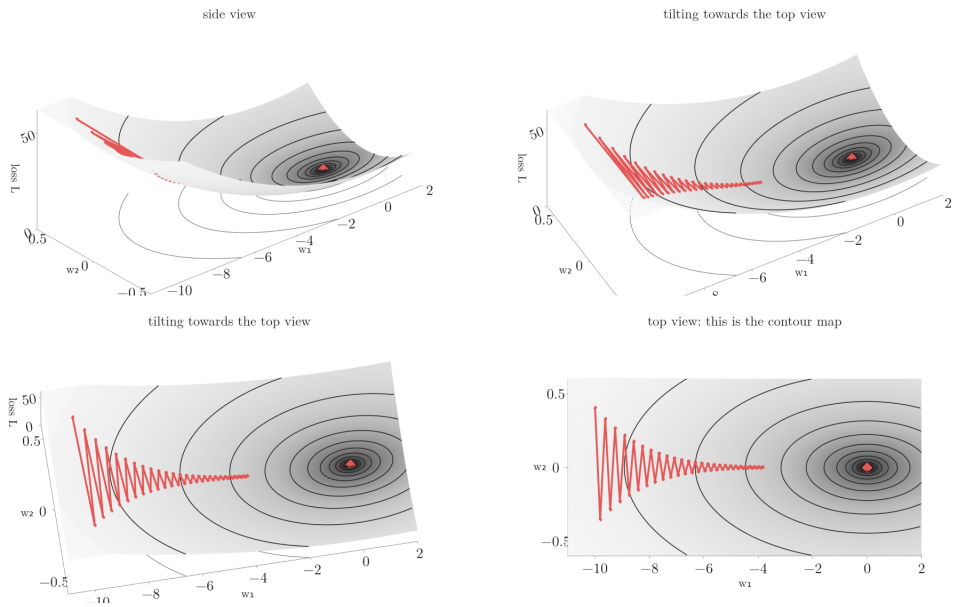


Figure 4: The valley as a surface, seen from the side and then tilted to the top view. Red: gradient descent with $\eta = 0.019$ from $(-10, 0.4)$, 50 steps. Height is the loss L. Thin lines join points of equal height, on the surface and dropped to the floor; each line is about double the one inside it. The walls go higher than drawn

Figure 4 shows the surface: a long, narrow valley, with steep walls on the two sides (w_2) and a floor that slopes gently down the length (w_1) to one lowest point (diamond). The height is the loss itself, and the surface is cut flat at $L = 62$: the walls go higher than drawn. The last frame is the **contour map** of this surface (how to read a contour map): the surface seen from above, where each line joins points at the same height. Lines close together mean a steep slope, so the lines are packed across the valley and spread out along it; the centre ring is the lowest point. Figure 5 below draws this same map for three learning rates.

The slopes are $\partial L/\partial w_1 = w_1$ and $\partial L/\partial w_2 = 100\, w_2$, so one gradient descent step is:

$$w_1 \leftarrow w_1 - \eta\, w_1 = (1 - \eta)\, w_1$$

$$w_2 \leftarrow w_2 - \eta \times 100\, w_2 = (1 - 100\eta)\, w_2$$

Every step multiplies w_1 by $1 - \eta$ and w_2 by $1 - 100\eta$. Three learning rates, with those two multipliers:

η	$1 - \eta$ (for w_1)	$1 - 100\eta$ (for w_2)	What happens
0.002	0.998	0.8	w_2 shrinks, w_1 barely moves

η	$1-\eta$ (for w_1)	$1-100\eta$ (for w_2)	What happens
0.019	0.981	-0.9	w_2 flips sign every step but shrinks; w_1 shrinks 1.9% per step
0.021	0.979	-1.1	w_2 flips sign and grows: divergence

For w_2 to shrink, the multiplier must lie between -1 and 1:

$$-1 < 1 - 100\eta < 1$$

$$0 < \eta < 0.02$$

So the steep direction caps η below 0.02, and at that cap w_1 shrinks by under 2% per step.

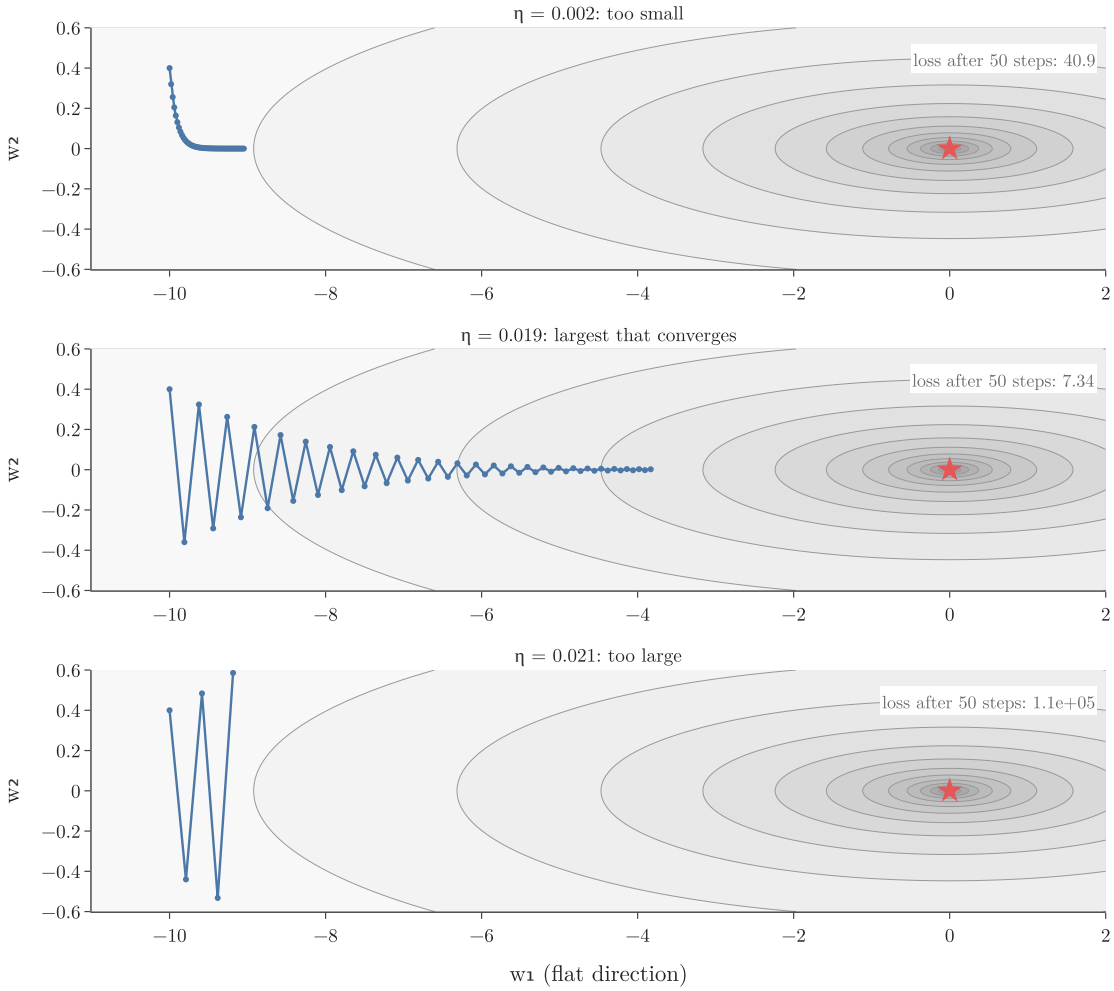


Figure 5: Gradient descent on the valley $L = (w_1^2 + 100w_2^2)/2$ from $(-10, 0.4)$, 50 steps each. Top: $\eta = 0.002$ barely moves along the valley. Middle: $\eta = 0.019$ zigzags across the valley and is still far from the minimum (star). Bottom: $\eta = 0.021$ bounces out of the valley

Each panel of Figure 5 is the contour map of Figure 4 with one path on it: every dot is the position after one step, starting at $(-10, 0.4)$, and the star is the lowest point. The rings are the lines of Figure 4, one for each doubling of the loss. The number in the corner is the loss after 50 steps.

Figure 5 and the Notebook show the trap. Steps needed to bring the loss below 0.01:

η	0.002	0.005	0.01	0.019	0.02 or more
Steps	2,128	850	424	223	never

Even the best learning rate needs 223 steps, all because the steep direction caps η while the flat direction needs large steps. Plain gradient descent cannot go fast in one direction and

slow in another.

5.4 Local minima

> **Key point:** A network's loss has many dips. **Gradient descent** (G-862) stops in the first one it reaches, which may be a local minimum with a worse loss than the global one.

The loss of a neural network is non-convex (a **non-convex function**, G-1333): it has several minima (what goes wrong with a non-convex loss). The best solution is the **global minimum**, the weights with the lowest loss of all. Gradient descent stops wherever the slope is zero, so starting from an unlucky point it settles in a **local minimum** (G-1110) and returns a sub-optimal solution. Stochastic gradient descent's zigzag gives it some chance to jump out; batch and mini-batch gradient descent have less (the path to the minimum).

5.5 Saddle points

> **Key point:** At a saddle point the surface rises in one direction and falls in another. The slope there is zero and nearly zero on a wide plateau around it, so the updates almost stop.

A **saddle point** (G-1718) is a point where the surface slopes up in one direction and down in another (what curvature says about the shape). At the saddle itself the gradient is zero, so the update leaves the weights unchanged:

$$w_{t+1} = w_t - \eta \times 0$$

$$= w_t$$

Saddle points are usually surrounded by a **plateau** (G-1503) where the gradient is close to zero in every direction, so **gradient descent** (G-862) crawls there for a long time even though the point is not a solution (Ruder 2016, §3).

Figure 6 shows the crawl on the simplest saddle, $L = w_1^2 - w_2^2$, which rises along w_1 and falls along w_2. Gradient descent starts at $(1.5, 0.001)$, almost exactly on the ridge, with learning rate 0.1.

In Figure 6 the left panel is the surface of $L(w_1, w_2) = w_1^2 - w_2^2$ with the path on it. The height at the start is:

$$L(1.5, 0.001) = 1.5^2 - 0.001^2$$

$$= 2.25 - 0.000001$$

≈ 2.25

The right panel plots the length of each step. A **log scale** axis: each gridline is 10 times the one below it, so equal vertical distances mean equal ratios, and a straight line down is a steady percentage drop per step. Watch the right panel fall while the left path nears the flat centre.

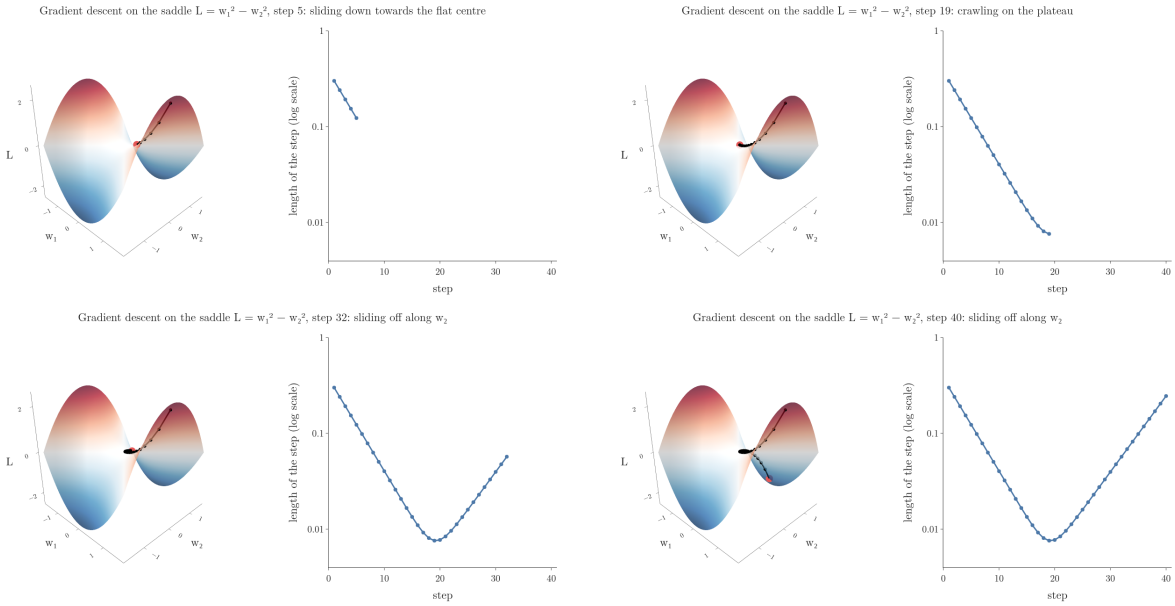


Figure 6: Gradient descent near a saddle point. Left: the surface $L = w_1^2 - w_2^2$ with the path (red dot: current position). Right: the length of each step, on a log scale

1. **Steps 1 to 10:** the path runs down the ridge towards the centre. Each step is shorter than the last, because the slope along w_1 shrinks.
2. **Around step 19:** the path is at the flat centre. The step length has fallen from 0.30 to 0.008, about 40 times shorter, and 17 of the 40 steps are shorter than a tenth of the first one.
3. **After step 30:** the small slope along w_2 has slowly grown, and the path slides off the saddle towards lower loss.

The saddle did not stop gradient descent, but most of the steps were spent on the plateau making almost no progress.

> **Extra:** For deep networks, saddle points may be a bigger obstacle than local minima: Dauphin et al. (2014) argue that in high dimensions most points with zero gradient are saddles, not minima (cited in Ruder 2016, §3; see also Goodfellow et al. 2016, §8.2.3).

5.6 What goes wrong as a result

> **Key point:** Training is either very slow, or it ends at a poor solution.

Because of these weak spots, plain gradient descent either trains slowly (a tiny learning rate, a flat direction, a plateau) or stops at a worse solution than it could reach (a **local minimum** (G-1110)). The optimizers that follow are designed to do better on both counts.

6. The optimizers ahead

> **Key point:** Momentum, NAG, AdaGrad, RMSProp and Adam. Each is a small change to the **gradient descent** (G-862) update, and most of them use an exponentially weighted moving average.

Five **optimizers** (G-1401) improve gradient descent along the two ideas of Figure 1:

Optimizer	Main idea	Weak spot it targets
Momentum (G-1258)	keep a running average of past gradients and build up speed	flat directions, narrow valleys, small local dips
Nesterov accelerated gradient (NAG) (G-1315)	momentum that looks ahead before it corrects	momentum's overshooting
AdaGrad (G-168)	a separate, shrinking learning rate for every parameter	one learning rate for every direction
RMSProp (G-1697)	AdaGrad that forgets old gradients	AdaGrad's learning rate shrinking to nothing
Adam (G-169)	momentum and RMSProp together	both

Momentum, RMSProp and Adam all keep an **exponentially weighted moving average** (G-735) (EWMA): a running average that gives recent values more weight. We study the EWMA first.

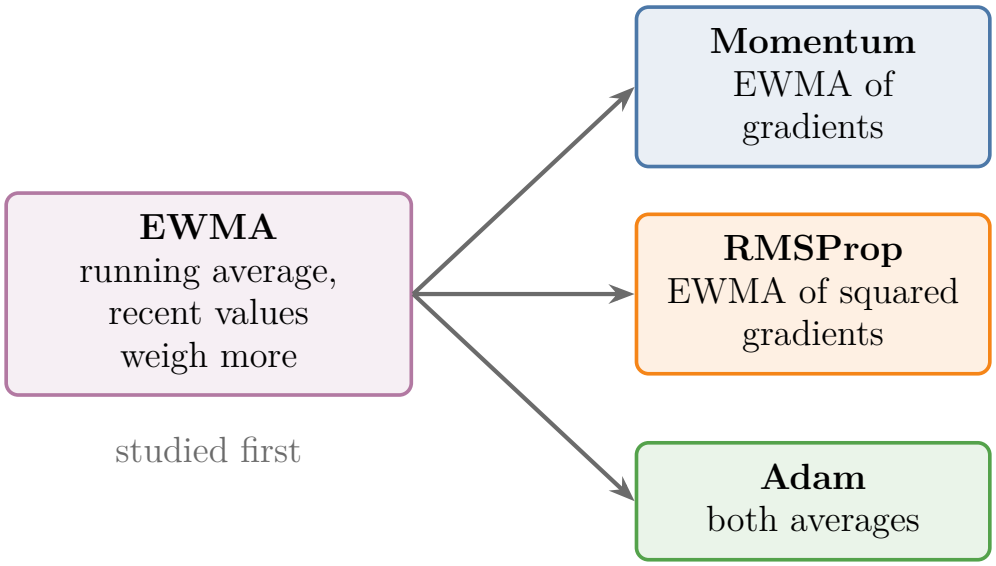


Figure 7: The EWMA is the shared tool: Momentum averages the gradients, RMSProp the squared gradients, and Adam keeps both averages

Figure 7 shows why the EWMA comes first: three of the five optimizers are built on it.

> **Extra:** Methods that use second derivatives, such as **Newton's method** (G-1321), can also handle steep and flat directions (second-order methods), but they are too expensive for the millions of parameters of a deep network (Ruder 2016, §4). The optimizers above use only the gradient.

7. Summary

Weak spot of gradient descent	What happens	Fixed by
Choosing the learning rate	too small: slow; too large: unstable	adaptive learning rates (AdaGrad, RMSProp, Adam)
Fixed schedules	cannot adapt to the dataset	adaptive learning rates
One learning rate for all directions	slow along flat directions of a narrow valley	momentum; per-parameter rates
Local minima	stops at a sub-optimal solution	momentum (can roll through small dips)
Saddle points	updates almost stop on the plateau	momentum, adaptive learning rates

- An **optimizer** (G-1401) finds the weights and biases that minimise the loss; gradient descent is the basic one.
- Batch, stochastic and mini-batch gradient descent differ only in the rows per update.
- The steep direction of a narrow valley caps the learning rate; the flat direction then crawls: 223 steps at best on our valley.
- The improved optimizers use two ideas: build up speed (**momentum** (G-1258), NAG) and adapt the learning rate (AdaGrad, RMSProp); Adam uses both.
- Most of them rely on the **exponentially weighted moving average** (G-735), so the EWMA is taught first.

8. Sources

Built from

- CampusX, "Optimizers in Deep Learning | Part 1 | Complete Deep Learning Course", YouTube, https://www.youtube.com/watch?v=iCTTnQJn50E

Other references

- Goodfellow, I., Bengio, Y. and Courville, A. (2016). *Deep Learning*. MIT Press. §8.2.3 (saddle points), §8.5 (algorithms with adaptive learning rates).
- Ruder, S. (2016). An overview of gradient descent optimization algorithms. arXiv:1609.04747. §3 (challenges), §4 (algorithms).
- Dauphin, Y., Pascanu, R., Gulcehre, C., Cho, K., Ganguli, S. and Bengio, Y. (2014). Identifying and attacking the saddle point problem in high-dimensional non-convex optimization. NeurIPS 2014.

9. Key terms

Terms taught in this Note come first; linked terms are recaps, taught in the Note the link opens.

Term	Meaning
Optimisation problem (G-1399)	Finding the inputs (here the weights and biases) that make a function (here the loss) smallest.
Saddle point (G-1718)	A flat point that curves up in one direction and down in another.
Learning rate scheduler (learning schedule) (G-1070)	A rule that changes the learning rate as training goes on, usually lowering it.
Initialisation (G-947)	Giving every weight and bias its starting value before training begins; a bad start can cause vanishing or exploding gradients or slow convergence.
Batch normalisation (G-266)	A layer placed between layers of a network that re-centres and re-scales its inputs during training; this keeps the activations away from the flat ends of sigmoid and tanh, which helps against vanishing gradients.
Activation function (G-165)	The function that turns a node's weighted sum z into its output, bringing it into a fixed range.
Optimizer (G-1401)	The rule that turns gradients into weight updates, such as plain gradient descent or Adam.
Gradient descent (G-862)	Finding the lowest point of a function by repeated small steps downhill.
Exponentially weighted moving average (EWMA) (G-735)	A running average in which recent values count most and older values count less and less, updated as $V_t = \beta V_{t-1} + (1-\beta)\theta_t$. Momentum and Adam use it to average the gradients.
Feature (G-772)	One piece of information about each example that a model uses (e.g. a student's CGPA).
Target (G-1949)	The output column we predict, such as the class; also called the label.
Global minimum (G-848)	The lowest point of the whole function.
Partial derivative (G-1457)	The slope of a function of several variables in one variable, holding the others fixed.
Gradient (of the loss) (G-863)	How fast the loss changes with each weight and bias, collected in one vector (the partial derivatives of the loss); it points in the direction in which the loss rises fastest.
Learning rate (η) (G-1068)	How strongly each update changes the model; in gradient descent, the number the slope is multiplied by to get the step size.

Term	Meaning
Epoch (G-696)	One full update of the parameters using the whole training set.
Convergence (MICE) (G-472)	The point where the fills hardly change between two iterations.
Non-convex function (G-1333)	A function whose curve is not one simple bowl: some straight line between two of its points (a chord) lies below part of the curve. It can have several local minima.
Local minimum (G-1110)	A point lower than everything around it, but not the lowest overall.
Plateau (G-1503)	A nearly flat region of the loss, where steps become very small.
Momentum (optimizer) (G-1258)	Gradient descent that keeps part of its past movement: each step follows an average of past gradients in which older ones count less and less (a velocity, an exponentially decaying average).
Nesterov accelerated gradient (NAG) (G-1315)	A version of momentum that first takes the momentum jump and then measures the gradient where it lands (the look-ahead point); seeing the slope ahead lets it brake before the minimum and swing less.
AdaGrad (G-168)	An optimizer that gives each parameter its own learning rate, $\eta/(\sqrt{v_t}+\epsilon)$, where v_t sums its past squared gradients.
RMSProp (G-1697)	An optimizer that divides each parameter's step by the root of a running average (EWMA) of its recent squared gradients, $v_t = \beta v_{t-1} + (1-\beta)g_t^2$; old gradients fade, so unlike AdaGrad the learning rate does not shrink to nothing.
Adam (G-169)	An optimizer, the rule that updates a network's weights to reduce the loss: a variant of gradient descent that keeps running averages of past gradients and of their squares, giving each weight its own step size; fairly robust to its settings, so a common default.

Term	Meaning
Newton's method (G-1321)	A way to find a minimum: at the current point, fit a bowl (the second-order Taylor polynomial, from the first and second derivatives), jump to its lowest point, and repeat; the step is $\boldsymbol{\delta} = -H^{-1}\nabla f^{\mathsf{T}}$. XGBoost uses one such step for every tree.